

1. I/O HARDWARE ARCHITECTURE & COMMUNICATION MECHANISMS

Device Controller & Registers

Electronic component on motherboard/expansion card bridging device and system bus:

- **Data-In Register:** Host reads data received from device.
- **Data-Out Register:** Host writes data to send to device.
- **Status Register:** Bits for *Busy*, *Ready*, *Error* flags.
- **Control Register:** Host writes commands (Read, Write, Seek).

Addressing: PMIO (Port I/O: special in/out CPU instructions) vs MMIO (Memory-Mapped I/O: registers mapped to RAM address space, accessed via standard pointers).

Polling vs Interrupt-Driven I/O

- **Polling (Busy-Waiting):** CPU repeatedly reads Status Register *Busy* bit in tight loop. Simple; acceptable for ultra-fast devices, but totally starves CPU on slow devices.
- **Interrupt-Driven I/O:** CPU initiates I/O and switches to another process. Device controller asserts Interrupt Request Line (IRL) when ready. CPU halts current instruction, saves context, indexes **Interrupt Vector Table (IVT)**, and invokes **Interrupt Service Routine (ISR)**.
- **Interrupt Hardware:** Non-Maskable (NMI: fatal hardware faults) vs Maskable (can be delayed during critical sections).

Direct Memory Access (DMA)

Specialized hardware bypasses CPU for high-speed block transfers (Disks, NICs):

- **Architecture:** CPU programs DMAC (DMA Controller) with Source Address, Destination Address, Transfer Count, and Command (R/W), then yields bus control.
- **Execution:** DMAC transfers entire data blocks directly between device buffer and physical RAM.
- **Cycle Stealing:** DMAC snatches memory bus cycles from CPU while CPU computes internally from L1/L2 cache.
- **Completion:** DMAC generates exactly **1 interrupt per block** (instead of 1 interrupt per byte).

2. KERNEL I/O SUBSYSTEM SERVICES & ABSTRACTIONS

I/O Scheduling, Buffering & Caching

- **I/O Scheduling:** OS reorders pending request queues to optimize physical device efficiency, minimize seek latency, and ensure fairness across processes.
- **Buffering:** Memory area holding data during transfer to resolve:
 - *Speed Mismatch:* Fast CPU vs slow modem/disk.
 - *Transfer Size Mismatch:* Network packets vs disk block chunks.
 - *Copy Semantics:* Preserving application data state at write() time.
 - *Schemes:* Single Buffering, Double Buffering (Ping-Pong: CPU processes buf 1 while device fills buf 2), Circular Buffering.
- **Caching vs Buffering:** Caching keeps copies of frequently read data in fast RAM; Buffering temporarily holds transit data between producer and consumer.

Spooling, Error Handling & Device Reservation

- **Spooling (Simultaneous Peripheral Operations On-Line):** Disk buffer holding output jobs for non-sharable / non-multiplexable devices (e.g. Printer, Tape drives). OS printer daemon consumes spool queue sequentially while user processes finish immediately without blocking.
- **Error Handling:** Transient errors (parity faults, transient seek errors) auto-retried with exponential backoff. Permanent errors propagated via system error codes (errno: EIO, ENOSPC).
- **I/O Protection:** All I/O instructions are privileged (Kernel Mode only). User applications invoke I/O strictly via System Calls (Trap to Kernel).
- **Device Streams:** UNIX STREAMS architecture enables modular pipelining of device drivers, protocol stacks, and line disciplines.

3. PHYSICAL MAGNETIC DISK GEOMETRY & ACCESS LATENCY EQUATIONS

Physical Architecture & Terminology

- **Platters & Spindle:** Stack of circular aluminum/glass platters coated with magnetic material rotating on a common spindle at constant angular velocity (5400, 7200, 10000, 15000 RPM).
- **Tracks & Sectors:** Each surface is divided into concentric rings called **Tracks**. Each track is partitioned into **Sectors** (standard 512 Bytes or Advanced Format 4 KB).
- **Cylinders:** Set of all tracks at identical radial distance across all platter surfaces. Data across a cylinder can be accessed without moving the mechanical arm!
- **Read/Write Heads:** Aerodynamically float on an air cushion (~nanometers above platter) mounted on a synchronized movable actuator arm.

Access Time Formulas & Solved Numerical

$$T_{\text{access}} = T_{\text{seek}} + T_{\text{rotational}} + T_{\text{transfer}} + T_{\text{controller}}$$

- **Seek Time (T_{seek}):** Time to move actuator arm to target cylinder (Dominant mechanical bottleneck: 3 - 9 ms).
- **Rotational Latency ($T_{\text{rotational}}$):** Time for target sector to rotate under head.

$$T_{\text{rot}}(\text{avg}) = 60 / (2 * \text{RPM}) \text{ sec} = 30000 / \text{RPM ms}$$
 For 7200 RPM: $T_{\text{rot}}(\text{avg}) = 30,000 / 7200 = 4.167 \text{ ms}$.
- **Transfer Time:** $T_{\text{trans}} = (\text{Bytes to transfer}) / (\text{Transfer Rate})$.
- **Controller Overhead:** Handshake & bus protocol delay (~0.1 - 0.5 ms).

AKTU Numerical: 7200 RPM, avg seek 6 ms, transfer rate 50 MB/s. Time to read 512 B sector?

$$T = 6 \text{ ms (seek)} + 4.167 \text{ ms (rot)} + (512 / 50 * 10^6 = 0.01 \text{ ms}) + 0.2 \text{ ms} = 10.377 \text{ ms}$$

1. DISK SCHEDULING ALGORITHMS MASTER BENCHMARK & COMPARISON

Standard Benchmark Parameters (AKTU PYQ): Request Queue = 98, 183, 37, 122, 14, 124, 65, 67 | Initial Head = 53 | Disk Range = 0 to 199 | Direction = Towards Higher Cylinders

Algorithm	Head Movement Trace Path	Seek Count	Core Pros	Core Cons
FCFS First-Come	53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67	640 cyl	100% Fair, Zero overhead, No starvation	Disastrous mechanical wild swings; worst seek count
SSTF Shortest Seek	53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183	236 cyl	High throughput, greedily minimizes seek time	Starvation of edge/distant requests under heavy load
SCAN Elevator	53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 37 → 14	236 cyl	Prevents starvation, uniform mechanical sweep	Always travels to disk boundary (199); favors middle
C-SCAN Circular SCAN	53 → ... → 183 → 199 → 0 (no service) → 14 → 37	382 cyl	Uniform wait time across all cylinders	Wastes seek time on full return sweep (199 → 0)
LOOK Boundary Opt.	53 → 65 → 67 → 98 → 122 → 124 → 183 → 37 → 14	208 cyl (Best)	Optimal! Reverses at last request; skips boundaries	Direction state tracking overhead in kernel
C-LOOK Circular LOOK	53 → ... → 183 → 14 (jumps to lowest request) → 37	322 cyl	Uniform waiting without traversing dead ends	Higher seek than LOOK due to return hop

2. RAID ARCHITECTURE (REDUNDANT ARRAY OF INDEPENDENT DISKS)

1. Striping (Performance)

Data split into consecutive chunks/stripes spread across multiple disks in parallel. Increases read/write throughput via multi-channel parallelism. Provides 0 fault tolerance.

2. Mirroring (Redundancy)

Identical copy of data written simultaneously to duplicate mirror disk. 100% data protection against single disk failure; fast read load balancing. 50% storage overhead.

3. Parity (Error Recovery)

Bitwise XOR error-correcting code ($P = D1 \oplus D2 \oplus D3$). If any disk fails: $D1 = P \oplus D2 \oplus D3$. Saves storage vs mirroring.

RAID Level	Architecture & Parity Strategy	Usable Capacity	Min Disks	Fault Tolerance	Write Penalty & Primary Use Case
RAID 0	Block-level Striping without redundancy	$N \times S$ (100%)	2	0 disks (Data loss)	0 penalty. High-speed video editing, scratch disks.
RAID 1	Disk Mirroring (Shadowing)	$1 \times S$ (50% if $N=2$)	2	$N - 1$ disks	2× writes. OS system boot drives, critical DB logs.
RAID 2	Bit-level Striping + Hamming Code ECC	$(N - ECC) \times S$	3	1 disk	Heavy controller ECC overhead. Obsolete.
RAID 3	Byte-level Striping + Dedicated Parity disk	$(N - 1) \times S$	3	1 disk	Dedicated parity disk is heavy I/O bottleneck.
RAID 4	Block-level Striping + Dedicated Parity disk	$(N - 1) \times S$	3	1 disk	Parity disk bottleneck on small concurrent writes.
RAID 5	Block-level Striping + Distributed Parity	$(N - 1) \times S$	3	1 disk failure	4 I/Os write penalty (2 reads + 2 writes). Enterprise servers.
RAID 6	Dual Distributed Parity (P + Q Reed-Solomon)	$(N - 2) \times S$	4	2 disks failure	6 I/Os write penalty (3 reads + 3 writes). Large archival arrays.
RAID 10 (1+0)	Striped set of Mirrored pairs	$(N / 2) \times S$	4	1 per mirrored pair	High performance & faster rebuild than RAID 0+1. Production DBs.

RAID 5 Write Penalty Derivation: Updating 1 block requires: Read old data, Read old parity, Compute new parity ($P_{new} = P_{old} \oplus D_{old} \oplus D_{new}$), Write new data, Write new parity = 2 Reads + 2 Writes = 4 Disk I/O operations.

1. FILE CONCEPT, DIRECTORY SYSTEMS & VIRTUAL FILE SYSTEM (VFS)

File Control Block (FCB) & Directories

- FCB / Inode:** File metadata container: Identifier, Type, Size, Permissions (rwx), Timestamps (atime, mtime, ctime), Owner ID, and Disk Block Pointers. Does NOT store file name in UNIX!
- Directory Structures:** Single-Level (name collision), Two-Level (user isolation), Hierarchical Tree (paths /a/b/c), Acyclic Graph (shared files via links).
- Hard Link:** New directory entry pointing to *existing Inode*. Increments i_nlink. Deleting original preserves data. Cannot cross filesystems.
- Soft (Symbolic) Link:** New file with *distinct Inode* containing destination path text string. Can cross filesystems; creates dangling link if target deleted.

Virtual File System (VFS) Architecture

Kernel abstraction layer decoupling generic POSIX system calls (open, read, write) from concrete filesystem implementations (ext4, NTFS, NFS, FAT):

- Superblock Object:** Entire mounted filesystem metadata (magic number, block size, total blocks, free blocks).
- Inode Object:** Individual file physical metadata and block pointers.
- Dentry Object:** Directory entry mapping human-readable string path names to Inode numbers; cached in fast RAM **dcache**.
- File Object:** Per-process open file instance (file offset pointer, access mode flags).

2. FILE ALLOCATION METHODS & UNIX INODE MATHEMATICAL DERIVATION

Method	Block Layout Scheme	Access Performance	Fragmentation	Key Trade-off
Contiguous	Sequential contiguous disk blocks	Fast Sequential & O(1) Direct	External Fragmentation	File size cannot grow dynamically without relocation
Linked	Each block stores pointer to next block	Sequential only; O(N) Direct	Zero External; internal only	Slow seek traversal; 1 corrupted pointer destroys rest of file
FAT	Pointers stored in dedicated RAM table	Fast Linked traversal in RAM	Zero External	Entire FAT table must reside in physical RAM (memory cost)
Indexed	Index block stores pointers to all blocks	O(1) Direct & Fast Sequential	Zero External	Index block overhead for small files; requires multilevel index

UNIX Inode Sizing Math (AKTU 10-Marks)

Inode has 12 Direct, 1 Single, 1 Double, 1 Triple Indirect pointer.

Pointers/Block $K = \text{Block_Size} / \text{Pointer_Size}$

Given: Block = 4 KB, Pointer = 4 B $\Rightarrow K = 4096 / 4 = 1024 (2^{10})$

- Direct: $12 \times 4 \text{ KB} = \mathbf{48 \text{ KB}}$
- Single Indirect: $1 \times 1024 \times 4 \text{ KB} = \mathbf{4 \text{ MB}}$
- Double Indirect: $1 \times (1024)^2 \times 4 \text{ KB} = \mathbf{4 \text{ GB}}$
- Triple Indirect: $1 \times (1024)^3 \times 4 \text{ KB} = \mathbf{4 \text{ TB}}$

Max File Size = 48 KB + 4 MB + 4 GB + 4 TB $\approx$ 4 TB

Free Space Management Schemes

- Bit Vector / Bitmap:** 1 bit per block (0=Alloc, 1=Free). Block# = (0-words $\times$ 32) + offset. CPU bsf/ffs instructions give O(1) allocation.
RAM Bytes = Disk_Size / (8 * Block_Size)
1.3 TB Disk with 512 B block = 302.68 MB RAM! (4KB block = 37.8 MB)
- Linked Free List:** Pointers inside free blocks. Zero space waste, but disastrous seek traversal.
- Grouping:** 1st free block holds (N-1) free block pointers + 1 pointer to next group block. 1 read yields N-1 blocks.
- Counting:** Stores tuples (start_block, count) for contiguous extents; stored in B-Tree (ext4, XFS).

3. FILE PROTECTION & ACCESS MATRIX MODEL

Access Matrix: ACL vs Capability List

Theoretical 2D Matrix of Domains (Rows) $\times$ Objects (Columns):

- Access Control Lists (ACL):** Decompose matrix by **Columns**. List attached to File ($F1 \rightarrow [(UserA, rw), (UserB, r)]$).
Pros: Trivial to revoke rights. **Cons:** Slower lookup on every open. Used in Windows NTFS, POSIX ACLs.
- Capability Lists (C-Lists):** Decompose matrix by **Rows**. Ticket/Token badge attached to Process ($ProcA \rightarrow [(F1, rw), (Printer, print)]$).
Pros: O(1) instant verification. **Cons:** Extremely hard revocation. Used in seL4 microkernel.

UNIX 9-Bit Octal Permissions (chmod 754)

Divided into 3 classes: **Owner (u), Group (g), Others (o):**

r = 4 (Read), w = 2 (Write), x = 1 (Execute)

- chmod 754:**
 - Owner: $4 + 2 + 1 = \mathbf{7}$ (rwx - full access)
 - Group: $4 + 0 + 1 = \mathbf{5}$ (r-x - read & execute)
 - Others: $4 + 0 + 0 = \mathbf{4}$ (r-- - read only)
- Special Modes:** SUID (4000: runs as owner, e.g. passwd), SGID (2000), Sticky Bit (1000: only owner can delete, e.g. /tmp 1777).